

5.1 Toma de Decisiones: `if`, `else if`, `else` y Expresiones Condicionales

En Dart, las estructuras condicionales evalúan expresiones de tipo `bool` estricto (no se permiten conversiones implícitas de enteros o punteros a booleanos).

```
int bateria = 15;

if (bateria <= 10) {
  print('Modo ahorro crítico');
} else if (bateria <= 20) {
  print('Advertencia de batería baja');
} else {
  print('Nivel de batería adecuado');
}
```

Operador Ternario

Para asignaciones condicionales concisas de una sola línea, el operador ternario `condicion ? expresion1 : expresion2` es la opción idónea:

```
String estadoRed = estaConectado ? 'En línea' : 'Sin conexión';
```

5.2 Control de Flujo Multicamino: `switch` y *Pattern Matching* en Dart 3+

Con la llegada de Dart 3, la sentencia `switch` evolucionó de ser un simple selector de valores constantes a una potente herramienta de **Pattern Matching** y **Switch Expressions**.

Switch Tradicional vs. Switch Expression

El `switch` tradicional ejecuta bloques de código basados en coincidencias:

```
String rol = 'admin';

switch (rol) {
  case 'admin':
    print('Acceso total');
    break;
  case 'user':
    print('Acceso limitado');
    break;
  default:
    print('Acceso denegado');
}
```

```
}
```

Por otro lado, una **Switch Expression** retorna un valor directamente, eliminando la necesidad del **break** explícito y haciendo el código significativamente más conciso:

```
final String permisos = switch (rol) {  
  'admin' => 'Acceso total',  
  'user'  => 'Acceso limitado',  
  _       => 'Acceso denegado', // '_' actúa como wildcard / default  
};
```

5.3 Estructuras Iterativas: **for**, **for-in**, **forEach**, **while** y **do-while**

Dart ofrece múltiples mecanismos para iterar sobre colecciones de datos o ejecutar bloques de código de forma repetitiva:

- **for Tradicional:** Controlado por un índice numérico explícito. Útil cuando se requiere conocer o manipular la posición exacta del elemento.
- **for-in:** La forma idónea y más limpia para recorrer elementos de una lista o conjunto (**Iterable**).
- **.forEach():** Método de orden superior que ejecuta una función anónima por cada elemento.
- **while:** Evalúa la condición antes de ejecutar el bloque. Si la condición es falsa desde el inicio, el bloque nunca se ejecuta.
- **do-while:** Garantiza que el bloque de código se ejecute **al menos una vez** antes de evaluar la condición.

5.4 Sentencias de Salida y Transferencia: **break**, **continue** y Etiquetas (**Labels**)

- **break:** Interrumpe y sale de manera inmediata del bucle o **switch** en ejecución.
- **continue:** Omite el resto de las instrucciones de la iteración actual y salta directamente a la siguiente evaluación de la condición del bucle.
- **Etiquetas (Labels):** Permiten identificar un bucle específico para romperlo o continuar desde bucles anidados profundos.

```
bucleExterior:  
for (int i = 0; i < 3; i++) {  
  for (int j = 0; j < 3; j++) {  
    if (i == 1 && j == 1) {  
      break bucleExterior; // Sale completamente del bucle externo 'i'  
    }  
  }  
}
```

```
}
```

5.5 Ejemplo

Creen el archivo `bin/d5_estructuras_control.dart` en VS Code. El siguiente programa simula un procesador de peticiones HTTP en un servidor local, utilizando estructuras condicionales, *Switch Expressions*, ciclos y transferencias de flujo.

```
/// Módulo D5: Estructuras de Control de Flujo
/// Demostración práctica de if-else, switch expressions, bucles iterativos y
etiquetas.

void main() {
  print('=====');
  print('      PROCESADOR DE PETICIONES Y CONTROL DE FLUJO - ESCOM      ');
  print('=====');

  // 1. Uso de Switch Expression (Dart 3+) para clasificar códigos HTTP
  final List<int> codigosRespuesta = [200, 404, 500, 401, 201, 503];

  print('--- 1. Clasificación de Códigos de Respuesta HTTP ---');
  for (final codigo in codigosRespuesta) {
    final String Categoria = switch (codigo) {
      >= 200 && < 300 => 'ÉXITO ($codigo)',
      >= 400 && < 500 => 'ERROR DEL CLIENTE ($codigo)',
      >= 500 && < 600 => 'ERROR DEL SERVIDOR ($codigo)',
      _                => 'CÓDIGO DESCONOCIDO ($codigo)',
    };
    print('  Petición evaluada -> $Categoria');
  }

  // 2. Control de Flujo con for-in, continue y break
  print('\n--- 2. Filtrado de Usuarios Activos en Lote ---');
  final List<Map<String, dynamic>> usuarios = [
    {'id': 101, 'nombre': 'Ana', 'activo': true, 'rol': 'admin'},
    {'id': 102, 'nombre': 'Carlos', 'activo': false, 'rol': 'user'}, // Se omitirá
    {'id': 103, 'nombre': 'Beatriz', 'activo': true, 'rol': 'editor'},
    {'id': 104, 'nombre': 'Daniel', 'activo': true, 'rol': 'banned'}, // Detendrá
    el proceso
    {'id': 105, 'nombre': 'Elena', 'activo': true, 'rol': 'user'},
  ];

  for (final usuario in usuarios) {
    // Si el usuario está inactivo, omitimos su procesamiento con continue
    if (usuario['activo'] == false) {
      print('  [OMITIDO] Usuario ID ${usuario['id']} está inactivo.');
```

```
// Si detectamos un rol de seguridad bloqueado, detenemos la cola con break
if (usuario['rol'] == 'banned') {
    print(' [ALERTA SEGURIDAD] Usuario bloqueado detectado
    (${usuario['nombre']}). Abortando cola.');
```

```
    break;
}

print(' [PROCESADO] Usuario: ${usuario['nombre']} | Rol: ${usuario['rol']}');
}

// 3. Demostración de do-while para intentos de reconexión
print('\n--- 3. Simulación de Intentos de Conexión (do-while) ---');
int intentoActual = 0;
final int maxIntentos = 3;
bool conexionExitosa = false;

do {
    intentoActual++;
    print(' Intentando conectar al servidor... Intento $intentoActual de
$maxIntentos');

    if (intentoActual == 2) {
        conexionExitosa = true;
        print(' -> ¡Conexión establecida exitosamente!');
    }
} while (!conexionExitosa && intentoActual < maxIntentos);

// 4. Uso de etiquetas (Labels) en bucles anidados
print('\n--- 4. Búsqueda Matricial con Etiquetas (Labels) ---');
final List<List<int>> matrizDatos = [
    [10, 20, 30],
    [40, 99, 60], // Contiene el valor objetivo '99'
    [70, 80, 90]
];

const int valorBuscado = 99;
bool encontrado = false;

busquedaMatriz:
for (int fila = 0; fila < matrizDatos.length; fila++) {
    for (int columna = 0; columna < matrizDatos[fila].length; columna++) {
        if (matrizDatos[fila][columna] == valorBuscado) {
            print(' Valor $valorBuscado encontrado en Fila: $fila, Columna:
$columna');
            encontrado = true;
            break busquedaMatriz; // Rompe el bucle exterior directamente
        }
    }
}

if (!encontrado) {
    print(' Valor $valorBuscado no encontrado en la matriz.');
```

```
print('\n=====');  
}
```

Instrucciones de Ejecución en VS Code:

```
dart run bin/d5_estructuras_control.dart
```

5.6 Ejercicios Prácticos

Implementen los siguientes dos programas en su entorno local para consolidar la manipulación de estructuras de control y *Switch Expressions*.

Ejercicio 1: Menu Interactivo de Gestion Académica

Crea un programa interactivo en Dart que simule un menú de consola utilizando un ciclo `do-while` y una sentencia `switch`:

1. Muestra un menú con las siguientes opciones:
 - 1. Ver estado del semestre
 - 2. Calcular promedio
 - 3. Salir
2. Solicita al usuario ingresar una opción por consola con `stdin.readLineSync()`.
3. Evalúa la opción utilizando un `switch` tradicional o *Switch Expression*.
4. El programa debe repetirse hasta que el usuario ingrese la opción 3.

Ejercicio 2: Evaluador Multicriterio de Riesgo Académico

Escribe un script en Dart que analice una lista de estudiantes representados por objetos `Map<String, dynamic>`:

1. Cada mapa contiene: `'nombre'`, `'asistencia'` (porcentaje `int`), y `'promedio'` (`double`).
 2. Recorre la lista con un bucle `for-in`.
 3. Utiliza una **Switch Expression** o estructuras `if-else` complejas para asignar un dictamen:
 - Asistencia < 80%: "REPROBADO POR FALTAS"
 - Promedio < 6.0: "REPROBADO POR CALIFICACIÓN"
 - Promedio >= 8.0 y Asistencia >= 90%: "EXCELENCIA ACADÉMICA"
 - En cualquier otro caso: "REGULAR"
 4. Imprime el dictamen final formateado de cada estudiante.
-